

ACTIVIDAD 3

Descomposición de Valores Singulares (SVD)

Reducción de dimensiones con datos de estudiantes

Gael García

1. Objetivo de la actividad

En esta actividad se aplicó la descomposición de valores singulares (SVD) a una base con calificaciones de 10 estudiantes. El objetivo fue reducir las cinco materias originales a representaciones de 2 y 3 dimensiones, visualizar a los estudiantes y comparar el error de reconstrucción en ambos casos.

2. Base de datos

La base contiene 10 estudiantes y cinco materias: Matemáticas, Ciencias, Español, Historia y Educación Física. La columna Nombre se utilizó solamente para identificar a cada estudiante en las gráficas.

Nombre	Matemáticas	Ciencias	Español	Historia	Ed. Física
Lucia	7.0	6.5	9.2	8.6	8.0
Pedro	7.5	9.4	7.3	7.0	7.0
Ines	7.6	9.2	8.0	8.0	7.5
Luis	5.0	6.5	6.5	7.0	9.0
Andres	6.0	6.0	7.8	8.9	7.3
Ana	7.8	9.6	7.7	8.0	6.5
Carlos	6.3	6.4	8.2	9.0	7.2
Jose	7.9	9.7	7.5	8.0	6.0
Sonia	6.0	6.0	6.5	5.5	8.7
Maria	6.8	7.2	8.7	9.0	7.0

3. Preparación de los datos

3.1 Librerías y carga del archivo

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = pd.read_csv("Estudiantes_SVD.csv")
data
```

El archivo CSV se cargó correctamente y se mostraron los 10 estudiantes con sus cinco calificaciones.

```
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

[5]: data = pd.read_csv("Estudiantes_SVD.csv")
data
```

	Nombre	Matematicas	Ciencias	Espanol	Historia	EdFisica
0	Lucia	7.0	6.5	9.2	8.6	8.0
1	Pedro	7.5	9.4	7.3	7.0	7.0
2	Ines	7.6	9.2	8.0	8.0	7.5
3	Luis	5.0	6.5	6.5	7.0	9.0
4	Andres	6.0	6.0	7.8	8.9	7.3
5	Ana	7.8	9.6	7.7	8.0	6.5
6	Carlos	6.3	6.4	8.2	9.0	7.2
7	Jose	7.9	9.7	7.5	8.0	6.0
8	Sonia	6.0	6.0	6.5	5.5	8.7
9	Maria	6.8	7.2	8.7	9.0	7.0

Carga de la base de datos en Jupyter Notebook.

3.2 Separación de nombres y datos numéricos

```
nombres = data["Nombre"]

A = data.drop(["Nombre"], axis=1)
A
```

Se guardaron los nombres por separado y se creó la matriz A únicamente con las cinco materias. Esta matriz es la que se utilizó para aplicar SVD.

```
[6]: nombres = data["Nombre"]

A = data.drop(["Nombre"], axis=1)
A
```

	Matematicas	Ciencias	Espanol	Historia	EdFisica
0	7.0	6.5	9.2	8.6	8.0
1	7.5	9.4	7.3	7.0	7.0
2	7.6	9.2	8.0	8.0	7.5
3	5.0	6.5	6.5	7.0	9.0
4	6.0	6.0	7.8	8.9	7.3
5	7.8	9.6	7.7	8.0	6.5
6	6.3	6.4	8.2	9.0	7.2
7	7.9	9.7	7.5	8.0	6.0
8	6.0	6.0	6.5	5.5	8.7
9	6.8	7.2	8.7	9.0	7.0

Matriz A con las cinco variables numéricas.

4. Descomposición SVD

```
from numpy import diag
from numpy import zeros
from scipy.linalg import svd

U, s, VT = svd(A)

Sigma = zeros((A.shape[0], A.shape[1]))
Sigma[:A.shape[1], :A.shape[1]] = diag(s)

print("Valores singulares:")
print(s)
```

SVD separó la matriz A en las matrices U, Sigma y VT. Los valores singulares obtenidos fueron: 53.2134, 5.3563, 3.8056, 1.4690 y 0.4780. Los primeros valores son los que concentran mayor parte de la información utilizada en las reducciones.

```
[7]: from numpy import diag
      from numpy import zeros
      from scipy.linalg import svd

      U, s, VT = svd(A)

      Sigma = zeros((A.shape[0], A.shape[1]))
      Sigma[:A.shape[1], :A.shape[1]] = diag(s)

[8]: print("Valores singulares:")
      print(s)

      Valores singulares:
      [53.21335049  5.35631448  3.80560674  1.46904724  0.47799818]
```

Valores singulares obtenidos con SVD.

5. Reducción a 2 dimensiones

```
n_elements = 2

UReduced2 = U[:, :n_elements]
SigmaReduced2 = Sigma[:n_elements, :n_elements]
VTReduced2 = VT[:n_elements, :]
```

Se conservaron solamente los primeros dos componentes. Así, U quedó con 2 columnas, Sigma con tamaño 2 x 2 y VT con 2 filas.

```
[9]: n_elements = 2

      UReduced2 = U[:, :n_elements]

      SigmaReduced2 = Sigma[:n_elements, :n_elements]

      VTReduced2 = VT[:n_elements, :]

[10]: print("U reducida:")
       print(UReduced2)

       print("\nSigma reducida:")
       print(SigmaReduced2)

       print("\nVT reducida:")
       print(VTReduced2)

      U reducida:
      [[-0.33090478 -0.29844247]
       [-0.32079813  0.35590965]
       [-0.33887152  0.21475231]
       [-0.28631997 -0.34779685]
       [-0.30374777 -0.3046643 ]
       [-0.33301138  0.39608249]
       [-0.31300374 -0.24404278]
       [-0.328788   0.47553197]
       [-0.27389698 -0.26452216]
       [-0.32638301 -0.10871076]]

      Sigma reducida:
      [[53.21335049  0.]
       [ 0.         5.35631448]]

      VT reducida:
      [[-0.40556463 -0.45759727 -0.46123811 -0.47103299 -0.43761787]
       [ 0.30388113  0.70497323 -0.20845983 -0.2165757 -0.56595801]]
```

Matrices reducidas para trabajar con 2 dimensiones.

5.1 Reconstrucción aproximada con 2 dimensiones

```
AReduced2 = UReduced2.dot(SigmaReduced2.dot(VTReduced2))

print("Matriz A transformada:")
print(AReduced2)
```

Con las matrices reducidas se reconstruyó una aproximación de la matriz original. Aunque sigue teniendo cinco columnas, la información utilizada para reconstruirla proviene solamente de dos dimensiones.

```
[11]: AReduced2 = UReduced2.dot(SigmaReduced2.dot(VTReduced2))

      print("Matriz A transformada:")
      print(AReduced2)

      Matriz A transformada:
      [[6.65563617 6.93068907 8.45496902 8.64041633 8.61053016]
       [7.50259774 9.15546106 7.47627701 7.62801112 6.39154032]
       [7.66288835 9.06253486 8.0774837 8.24477427 7.24032869]
       [5.61309793 5.65867129 7.41578628 7.58014066 7.72189396]
       [6.05941975 6.24591247 7.7953721 7.96693512 7.99698132]
       [7.83156613 9.60455223 7.73118336 7.88753689 6.55416982]
       [6.35785122 6.70021013 7.95486409 8.12861624 8.02875678]
       [7.86973795 9.80171752 7.53881312 7.68951159 6.21497236]
       [5.48053658 5.67061811 7.01789357 7.17215282 7.18015541]
       [6.86687283 7.53702066 8.13213661 8.30697925 7.93006916]]
```

5.2 Matriz T y gráfica en 2 dimensiones

```
T2 = UReduced2.dot(SigmaReduced2)

print("Matriz T (2 Dimensiones):")
print(T2)
```

La matriz T2 dejó a cada estudiante representado por dos valores. Estos dos valores se usaron como coordenadas para la gráfica.

```
[12]: T2 = UReduced2.dot(SigmaReduced2)

print("Matriz T (2 Dimensiones):")
print(T2)

Matriz T (2 Dimensiones):
[[-17.60855193 -1.59855173]
 [-17.07074326  1.90636402]
 [-18.03248887  1.15028089]
 [-15.23604486 -1.86290932]
 [-16.1634327  -1.6318778 ]
 [-17.7206514  2.12154237]
 [-16.65597775 -1.30716988]
 [-17.49591103  2.54709878]
 [-14.57497601 -1.41686385]
 [-17.36793336 -0.58228904]]
```

Matriz T2 de tamaño 10 x 2.

```
plt.figure(figsize=(8,6))
plt.scatter(T2[:,0], T2[:,1])

for i, nombre in enumerate(nombres):
    plt.annotate(nombre, (T2[i,0], T2[i,1]))

plt.title("SVD en 2 dimensiones")
plt.xlabel("Dimensión 1")
plt.ylabel("Dimensión 2")
plt.grid()
plt.show()
```

En la gráfica 2D se observan estudiantes con posiciones cercanas y otros más separados. Por ejemplo, Ana, José, Pedro e Inés aparecen en la parte superior, mientras que Luis y Sonia se encuentran más alejados hacia la parte inferior.

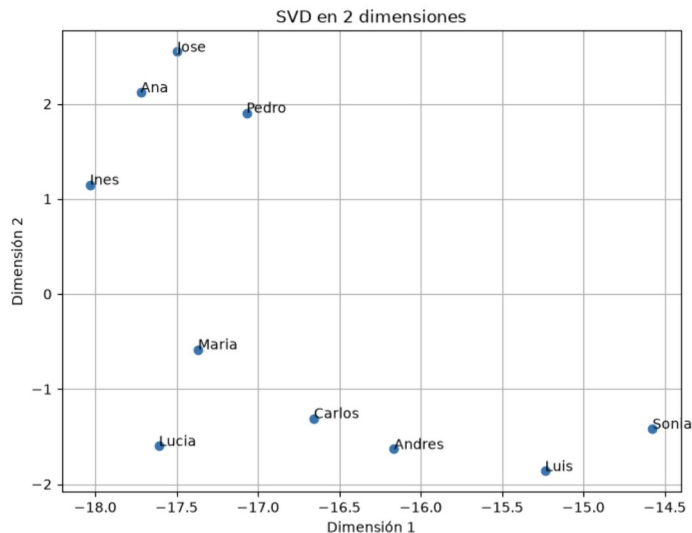
```
[13]: plt.figure(figsize=(8,6))

plt.scatter(T2[:,0], T2[:,1])

for i, nombre in enumerate(nombres):
    plt.annotate(nombre, (T2[i,0], T2[i,1]))

plt.title("SVD en 2 dimensiones")
plt.xlabel("Dimensión 1")
plt.ylabel("Dimensión 2")
plt.grid()

plt.show()
```



Representación de los 10 estudiantes en 2 dimensiones.

6. Reducción a 3 dimensiones

```
n_elements = 3

UReduced3 = U[:, :n_elements]
SigmaReduced3 = Sigma[:n_elements, :n_elements]
VTReducd3 = VT[:, :n_elements, :]

AReducd3 = UReduced3.dot(SigmaReduced3.dot(VTReducd3))

print("Matriz A transformada:")
print(AReducd3)
```

Después se repitió el procedimiento conservando tres componentes. Esto genera una reconstrucción con mayor información que la realizada con dos dimensiones.

```
[14]: n_elements = 3

UReduced3 = U[:, :n_elements]

SigmaReduced3 = Sigma[:n_elements, :n_elements]

VTReducd3 = VT[:, :n_elements, :]

AReducd3 = UReduced3.dot(SigmaReduced3.dot(VTReducd3))

print("Matriz A transformada:")
print(AReducd3)

Matriz A transformada:
[[6.65245886 6.72762726 8.69042319 9.08512051 8.09898421]
 [7.50649243 9.40437093 7.1876611 7.08289996 7.01858503]
 [7.66453175 9.1675645 7.95569976 8.01475998 7.50491554]
 [5.6205042 6.13200674 6.8669445 6.54353881 8.91430342]
 [6.05471948 5.94551744 8.14368606 8.62479847 7.24023707]
 [7.83115817 9.57847935 7.76141538 7.94463634 6.48848799]
 [6.35253602 6.36051578 8.34874639 8.87254488 7.17301112]
 [7.86825556 9.7069775 7.64866605 7.89699168 5.97630674]
 [5.49031524 6.29557293 6.29324613 5.80350542 8.75451809]
 [6.86123771 7.17687926 8.54972766 9.09568679 7.02281405]]
```

Matriz A reconstruida con 3 dimensiones.

6.1 Matriz T y gráfica en 3 dimensiones

```
T3 = UReduced3.dot(SigmaReduced3)

print("Matriz T (3 Dimensiones):")
print(T3)
```

La matriz T3 representa a cada estudiante con tres coordenadas.

```
[15]: T3 = UReduced3.dot(SigmaReduced3)

print("Matriz T (3 Dimensiones):")
print(T3)

Matriz T (3 Dimensiones):
[[-17.60855193 -1.59855173 0.74573717]
 [-17.07074326 1.90636402 -0.91411251]
 [-18.03248887 1.15028089 -0.38571754]
 [-15.23604486 -1.86290932 -1.73830731]
 [-16.1634327 -1.6318778 1.10318989]
 [-17.7206514 2.12154237 0.09575169]
 [-16.65597775 -1.30716988 1.24751522]
 [-17.49591103 2.54709878 0.34792931]
 [-14.57497601 -1.41686385 -2.29512399]
 [-17.36793336 -0.58228904 1.32260625]]
```

Matriz T3 de tamaño 10 x 3.

```
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(9,7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(T3[:,0], T3[:,1], T3[:,2])

for i, nombre in enumerate(nombres):
    ax.text(T3[i,0], T3[i,1], T3[i,2], nombre)

ax.set_title('SVD en 3 dimensiones')
ax.set_xlabel('Dimensión 1')
ax.set_ylabel('Dimensión 2')
ax.set_zlabel('Dimensión 3')
plt.show()
```

La tercera dimensión agrega más detalle a la representación. En esta vista se observa con mayor claridad la separación de algunos estudiantes, especialmente Luis y Sonia respecto al resto.

```
[16]: from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(9,7))
ax = fig.add_subplot(111, projection='3d')

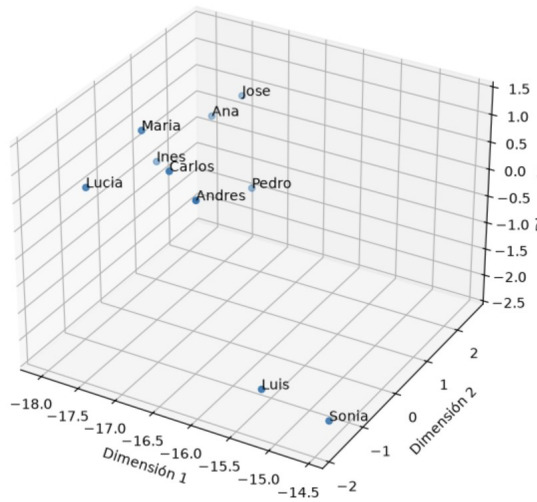
ax.scatter(T3[:,0], T3[:,1], T3[:,2])

for i, nombre in enumerate(nombres):
    ax.text(T3[i,0], T3[i,1], T3[i,2], nombre)

ax.set_title("SVD en 3 dimensiones")
ax.set_xlabel("Dimensión 1")
ax.set_ylabel("Dimensión 2")
ax.set_zlabel("Dimensión 3")

plt.show()
```

SVD en 3 dimensiones



Representación de los estudiantes en 3 dimensiones.

7. Error de reconstrucción

7.1 Error usando 2 dimensiones

```
Error2 = A.to_numpy() - AReduced2

plt.figure(figsize=(10,7))
dfError2 = pd.DataFrame(
    Error2,
    columns=["Matematicas", "Ciencias", "Espanol", "Historia", "EdFisica"]
)
dfError2.boxplot()
plt.title("Error de reconstrucción - 2 dimensiones")
plt.show()
```

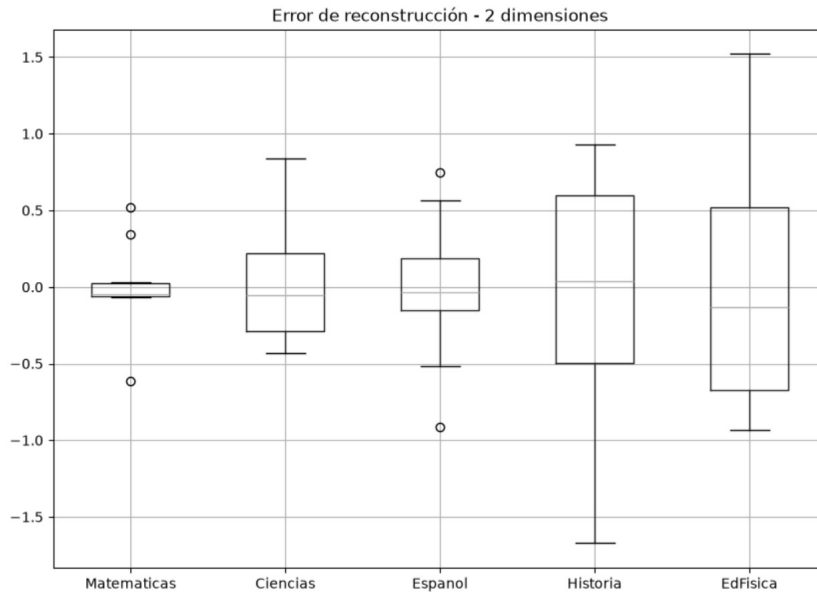
El boxplot muestra la distribución de los errores al reconstruir cada materia con dos dimensiones. Historia y Educación Física presentan una dispersión mayor que las demás materias.

```
[17]: # Error con 2 dimensiones
Error2 = A.to_numpy() - AReduced2

plt.figure(figsize=(10,7))

dfError2 = pd.DataFrame(
    Error2,
    columns=["Matematicas", "Ciencias", "Espanol", "Historia", "EdFisica"]
)

dfError2.boxplot()
plt.title("Error de reconstrucción - 2 dimensiones")
plt.show()
```



Errores de reconstrucción usando 2 dimensiones.

```
print("Errores promedio por materia:")
print(Error2.mean(axis=0))
```

Los errores promedio obtenidos fueron aproximadamente: Matemáticas -0.00002, Ciencias 0.01326, Español -0.01948, Historia -0.02451 y Educación Física 0.03306.

```
[18]: print("Errores promedio por materia:")
print(Error2.mean(axis=0))

Errores promedio por materia:
[-2.04650684e-05  1.32612606e-02 -1.94778869e-02 -2.45074289e-02
  3.30602005e-02]
```

Errores promedio por materia con 2 dimensiones.

7.2 Error usando 3 dimensiones

```
Error3 = A.to_numpy() - AReduced3

plt.figure(figsize=(10,7))
dfError3 = pd.DataFrame(
    Error3,
    columns=["Matematicas", "Ciencias", "Espanol", "Historia", "EdFisica"]
)
dfError3.boxplot()
plt.title("Error de reconstrucción - 3 dimensiones")
plt.show()
```

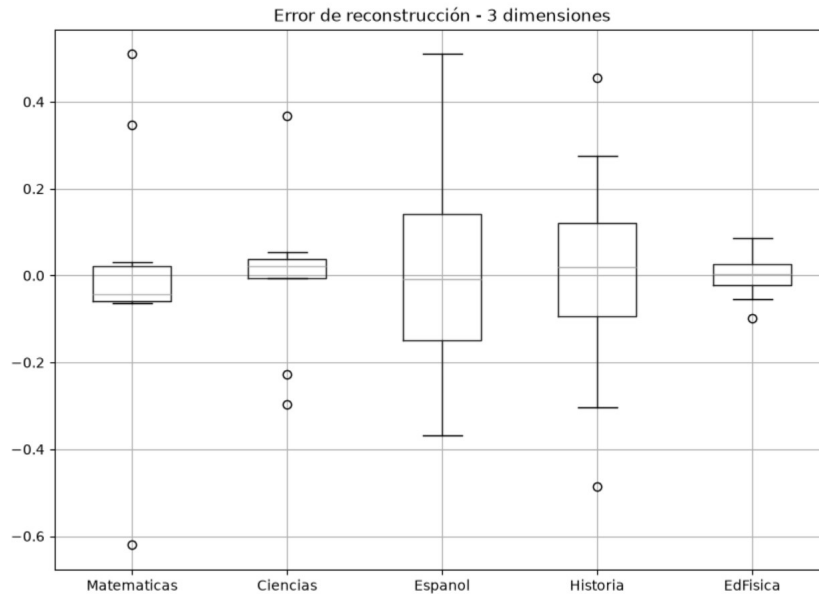
Con tres dimensiones, los errores aparecen más concentrados alrededor de cero en comparación con el caso de dos dimensiones.

```
[19]: # Error con 3 dimensiones
Error3 = A.to_numpy() - AReduced3

plt.figure(figsize=(10,7))

dfError3 = pd.DataFrame(
    Error3,
    columns=["Matemáticas", "Ciencias", "Español", "Historia", "EdFísica"]
)

dfError3.boxplot()
plt.title("Error de reconstrucción - 3 dimensiones")
plt.show()
```



Errores de reconstrucción usando 3 dimensiones.

```
print("Errores promedio por materia:")
print(Error3.mean(axis=0))
```

Los errores promedio fueron aproximadamente: Matemáticas -0.00022, Ciencias 0.00045, Español -0.00462, Historia 0.00355 y Educación Física 0.00078.

```
[20]: print("Errores promedio por materia:")
print(Error3.mean(axis=0))

Errores promedio por materia:
[-0.00022094  0.00044883 -0.00462162  0.00355172  0.00078359]
```

Errores promedio por materia con 3 dimensiones.

8. Comparación de resultados

Materia	Error medio 2D	Error medio 3D
Matemáticas	-0.00002	-0.00022
Ciencias	0.01326	0.00045
Español	-0.01948	-0.00462
Historia	-0.02451	0.00355
Educación Física	0.03306	0.00078

En los boxplots se aprecia que los errores de la reconstrucción con 3 dimensiones tienen una dispersión menor. Esto indica que, para estos datos, la representación de 3 dimensiones conserva mejor la información original que la representación de 2 dimensiones.

9. Conclusión

Al aplicar SVD se redujeron las cinco materias originales a representaciones de 2 y 3 dimensiones. Con 2 dimensiones fue posible visualizar a los estudiantes en una gráfica sencilla y observar cuáles quedaban cerca o lejos entre sí. Al utilizar 3 dimensiones se agregó más detalle a la representación y los errores de reconstrucción quedaron más concentrados alrededor de cero. Por lo tanto, en esta actividad la reducción a 3 dimensiones conserva mejor la información de las calificaciones originales, mientras que la representación en 2 dimensiones es más fácil de visualizar.